

NES — MULTI-MODEL EXPANSION

Experiments · Code Changes · Expected Results

Mistral · Gemma · Qwen · Phi · DeepSeek · Falcon · Llama-3

CONFIDENTIAL RESEARCH DOCUMENT

1. Executive Summary

NES (Neural Embedding Steganography) currently operates exclusively on TinyLlama-1.1B — a 32-layer Llama-architecture model quantized to NF4 via BitsAndBytes. The system embeds AES-256-GCM encrypted payloads into quantization residuals using the QACI pipeline and recovers them with zero BER under clean conditions.

Expanding NES to other open-source model families requires solving three classes of problem:

- Architectural differences — layer count, module naming, attention variant
- Quantization differences — NF4 vs GPTQ vs AWQ vs GGUF vs FP8
- Statistical differences — residual magnitude, distribution, and SNR per family

This document defines the exact experiments to run, the precise code changes required in every relevant file, and the expected result for each experiment. **Follow this plan in order. Do not run experiments in parallel — each experiment informs the next.**

2. Current State — What Is Hardcoded to TinyLlama

Before running any experiment on a new model, you must know exactly which parts of the codebase assume TinyLlama's architecture. The table below is a complete audit.

File	Hardcoded Assumption	Impact	Fix Required
src/carrier_intelligence/qaci_pipeline.py	total_layers=32 default	Wrong position bias for non-32-layer models	Auto-detect from model config
src/carrier_intelligence/layer_profiler.py	_position_bias() integer thresholds	Middle-layer preference breaks if layers != 32	Parameterise by total_layers fraction
src/carrier_intelligence/carrier_scheduler.py	gamma=2.5 default	Allocation tuned for 32-layer residual SNR	Tune per model family
src/embedding/intelligent_embedder.py	QACIPipeline(total_layers=32)	Hardcoded	Pass model.config.num_hidden_layers
scripts / model loading	TinyLlama-1.1B HuggingFace ID	Only loads one model	Build model registry
src/evaluation/fidelity_validator.py	PPL tested via TinyLlama tokenizer	Tokenizer mismatch for other models	Pass tokenizer as argument
Layer hook attachment	model.layers[j].mlp assumed	Gemma/Phi/Qwen/Falcon differ	Per-family name resolver
src/core/types.py EmbeddingConfig	No model_family field	Cannot dispatch model-specific logic	Add model_family + num_layers

3. Target Model Family Reference

Each model family is listed with its layer count, module naming convention, default quantization support, and key notes for NES adaptation.

Family	Representative Model	Layers	MLP Pattern	NF4 BnB	Notes
Llama-3	meta-llama/Llama-3.1-8B	32	model.layers[j].mlp	Yes	Closest to TinyLlama — minimal changes needed

Family	Representative Model	Layers	MLP Pattern	NF4 BnB	Notes
Mistral	mistralai/Mistral-7B-v0.3	32	model.layers[i].mlp	Yes	Sliding window attn; same module names as Llama
Mixtral MoE	mistralai/Mixtral-8x7B	32	model.layers[i].block_sparse_moe	Yes	MoE experts — residual structure differs
Gemma 2	google/gemma-2-9b	42	model.layers[i].mlp	Yes	42 layers — position bias must rescale
Qwen 2.5	Qwen/Qwen2.5-7B	28	model.layers[i].mlp	Yes	28 layers; GQA; similar residual stats to Mistral
Phi-3 Mini	microsoft/Phi-3-mini-4k-instruct	32	model.layers[i].mlp	Yes	Dense; 32 layers; good initial baseline
Phi-4	microsoft/phi-4	40	model.layers[i].mlp	Yes	40 layers; larger residuals than Phi-3
DeepSeek-R1	deepseek-ai/DeepSeek-R1-Distill-Llama-8B	32	model.layers[i].mlp	Yes	Llama distill — easiest port after Llama-3
Falcon	tiiuae/falcon-7b	32	transformer.h[i].mlp	Yes	Different root module — needs resolver
OLMo 2	allenai/OLMo-2-1124-7B	32	model.layers[i].mlp	Yes	Open weights; good for academic publications

4. Required Code Changes — Complete Specification

Make these changes before running any experiment. They are ordered from lowest to highest risk. Do not skip any.

4.1 EmbeddingConfig — Add Model Metadata (src/core/types.py)

Add two new fields so every downstream component can query the model architecture without importing transformers.

```
# src/core/types.py – ADD these fields to EmbeddingConfig dataclass

@dataclass
class EmbeddingConfig:
    total_payload_bits: int = 50_000
    embedding_strategy: str = 'sign'
    model_family: str = 'llama' # NEW
    num_hidden_layers: int = 32 # NEW – from model.config.num_hidden_layers
    alpha: float = 0.25
    gamma: float = 2.5
```

4.2 LayerProfiler — Dynamic Position Bias (src/carrier_intelligence/layer_profiler.py)

Replace the hardcoded integer thresholds with fractional breakpoints so any layer count works correctly.

```
# REPLACE _position_bias() with this version

def _position_bias(self, layer_id: int, total_layers: int) -> float:
    frac = layer_id / max(total_layers - 1, 1) # normalised 0.0 to 1.0
    if frac < 0.10: return 0.65 # very early layers (sensitive)
    elif frac < 0.20: return 0.80 # early layers
    elif frac < 0.80: return 1.00 # middle layers – full capacity
    elif frac < 0.90: return 0.80 # late layers
    else: return 0.65 # very late layers
```

4.3 QACIPipeline — Receive Layer Count from Config

Remove the hardcoded default of 32 and require explicit passing from the caller.

```
# src/carrier_intelligence/qaci_pipeline.py
# CHANGE __init__ signature – remove default value for total_layers

def __init__(self, total_layers: int, gamma: float = 2.5):
    self.total_layers = total_layers
    self.profiler = LayerProfiler()
    self.scheduler = CarrierScheduler(gamma=gamma)
```

4.4 IntelligentEmbedder — Use Config Fields

```
# src/embedding/intelligent_embedder.py
def __init__(self, config: EmbeddingConfig):
    self.config = config
    self.pipeline = QACIPipeline(
```

```
total_layers=config.num_hidden_layers,
gamma=config.gamma,
)
self.strategy = SignEmbeddingStrategy(config)
```

4.5 Model Registry — Module Name Resolver (src/models/registry.py — NEW FILE)

Create a new file that maps model family names to their layer module paths. This is the most important new file for multi-model support.

```
# src/models/registry.py

MODEL_REGISTRY = {
    'llama': { 'root': 'model.layers', 'mlp': '.mlp', 'attn': '.self_attn' },
    'mistral': { 'root': 'model.layers', 'mlp': '.mlp', 'attn': '.self_attn' },
    'gemma': { 'root': 'model.layers', 'mlp': '.mlp', 'attn': '.self_attn' },
    'qwen': { 'root': 'model.layers', 'mlp': '.mlp', 'attn': '.self_attn' },
    'phi': { 'root': 'model.layers', 'mlp': '.mlp', 'attn': '.self_attn' },
    'deepseek': { 'root': 'model.layers', 'mlp': '.mlp', 'attn': '.self_attn' },
    'olmo': { 'root': 'model.layers', 'mlp': '.mlp', 'attn': '.self_attn' },
    'falcon': { 'root': 'transformer.h', 'mlp': '.mlp', 'attn': '.self_attention' },
    'mixtral': { 'root': 'model.layers', 'mlp': '.block_sparse_moe', 'attn': '.self_attn' },
}

def get_layer_module(model, family: str, layer_id: int, component: str = 'mlp'):
    reg = MODEL_REGISTRY[family]
    root = model
    for part in reg['root'].split('.'):
        root = getattr(root, part)
    layer = root[layer_id]
    suffix = reg[component].lstrip('.')
    return getattr(layer, suffix)

def get_num_layers(model) -> int:
    cfg = model.config
    for attr in ('num_hidden_layers', 'n_layer', 'num_layers'):
        if hasattr(cfg, attr):
            return getattr(cfg, attr)
    raise ValueError('Cannot detect num_layers from model.config')
```

4.6 Universal Model Loader (src/models/loader.py — NEW FILE)

Universal loader handling any HuggingFace model with NF4 quantization, returning both quantized and FP16 reference models for residual extraction.

```
# src/models/loader.py
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig
import torch

NF4_CONFIG = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type='nf4',
    bnb_4bit_use_double_quant=True,
    bnb_4bit_compute_dtype=torch.float16,
)

def load_model_pair(model_id: str, device: str = 'cuda'):
    tokenizer = AutoTokenizer.from_pretrained(model_id, trust_remote_code=True)
```

```
nf4_model = AutoModelForCausalLM.from_pretrained(
    model_id, quantization_config=NF4_CONFIG,
    device_map=device, trust_remote_code=True,
)
fp16_model = AutoModelForCausalLM.from_pretrained(
    model_id, torch_dtype=torch.float16,
    device_map='cpu', trust_remote_code=True,
)
return nf4_model, fp16_model, tokenizer

def extract_residuals(nf4_model, fp16_model, family: str) -> dict:
    from src.models.registry import get_layer_module, get_num_layers
    n = get_num_layers(nf4_model)
    residuals = {}
    for i in range(n):
        nf4_mlp = get_layer_module(nf4_model, family, i, 'mlp')
        fp16_mlp = get_layer_module(fp16_model, family, i, 'mlp')
        nf4_w = nf4_mlp.down_proj.weight
        fp16_w = fp16_mlp.down_proj.weight.to(nf4_w.device)
        dq = nf4_w.dequantize().float() if hasattr(nf4_w, 'dequantize') else nf4_w.float()
        residuals[i] = (fp16_w.float() - dq).flatten()
    return residuals
```

4.7 Gamma Tuning Per Model Family

The gamma parameter controls how aggressively bits concentrate in high-quality layers. Tune per family based on residual profile from Experiment 2.

Model Family	Recommended gamma	Reasoning
Llama-3 / TinyLlama	2.5	Baseline — already tuned and validated
Mistral	2.5	Same architecture class; similar residual profile
Gemma 2	2.0	42 layers; flatter quality distribution — less concentration
Qwen 2.5	2.5	28 layers; similar SNR to Mistral
Phi-3 / Phi-4	3.0	Denser residuals; top layers much stronger — concentrate more
DeepSeek Distill	2.5	Llama base — same as Llama-3
Falcon	2.0	Different root module; flatter quality distribution
Mixtral MoE	1.5	Sparse MoE residuals — distribute bits more evenly

5. Experiments — Complete Specification

Run all experiments in the order given. Each has a clear prerequisite, setup, run procedure, and success criterion. Do not advance until the current experiment passes.

Experiment 1 — Architecture Probe

Goal: Confirm NES can attach hooks to the correct layers, extract residuals, and that QACI receives sensible inputs for any new model.

Run Procedure

```
# scripts/exp1_probe.py
from src.models.loader import load_model_pair, extract_residuals
from src.models.registry import get_num_layers
from src.carrier_intelligence.qaci_pipeline import QACIPipeline

MODEL_ID = 'mistralai/Mistral-7B-v0.3' # change per model
FAMILY = 'mistral'

nf4, fp16, tok = load_model_pair(MODEL_ID)
n_layers = get_num_layers(nf4)
print(f'Layers detected: {n_layers}')
```

```
residuals = extract_residuals(nf4, fp16, FAMILY)
for lid, r in sorted(residuals.items())[:5]:
    print(f' L{lid:02d} shape={r.shape} mean={r.abs().mean():.5f}')
```

```
pipeline = QACIPipeline(total_layers=n_layers)
result = pipeline.select(residuals, total_payload_bits=10_000)
print(f'QACI selected {result.total_selected} carriers across {n_layers} layers')
```

Expected Results

- Layer count matches architecture table in Section 3
- Mean absolute residual per layer: 0.001 – 0.02 for healthy NF4 models
- QACI pipeline completes and total_selected == 10,000

■ *Red flag: If mean residual < 0.0005 for most layers, the model may not quantize well to NF4. Try `bnb_4bit_quant_type='fp4'` as alternative.*

Pass Criterion

All layers probed, residuals non-zero, QACI allocates correct total bits. Save residual stats — needed for Experiment 3.

Experiment 2 — Residual Distribution Fingerprinting

Goal: Characterize the per-model residual distribution vs TinyLlama baseline. Determines whether alpha and gamma need re-tuning for each family.

Run Procedure

```
# scripts/exp2_residual_fingerprint.py
from src.carrier_intelligence.layer_profiler import LayerProfiler
import json

for MODEL_ID, FAMILY in [
    ('TinyLlama/TinyLlama-1.1B-Chat-v1.0', 'llama'),
    ('mistralai/Mistral-7B-v0.3', 'mistral'),
```

```
( 'google/gemma-2-9b', 'gemma'),
( 'Qwen/Qwen2.5-7B', 'qwen'),
]:
nf4, fp16, _ = load_model_pair(MODEL_ID)
n_layers = get_num_layers(nf4)
residuals = extract_residuals(nf4, fp16, FAMILY)
profiler = LayerProfiler()
profiles = [profiler.profile(residuals[i], i, total_layers=n_layers)
for i in sorted(residuals)]
stats = {
'model': MODEL_ID, 'num_layers': n_layers,
'mean_quality': sum(p['quality_score'] for p in profiles)/len(profiles),
'mean_mag_mean': sum(p['mag_mean'] for p in profiles)/len(profiles),
'mean_entropy': sum(p['entropy'] for p in profiles)/len(profiles),
}
print(json.dumps(stats, indent=2))
with open(f'residual_profile_{FAMILY}.json', 'w') as f:
json.dump(**stats, 'per_layer': profiles, f, indent=2)
```

Expected Results by Model

Model Family	Expected mag_mean Range	Expected entropy Range	Action if Outside Range
TinyLlama (baseline)	0.004 – 0.010	0.45 – 0.75	Baseline — no action needed
Mistral-7B	0.005 – 0.015	0.50 – 0.80	If entropy < 0.40: increase alpha to 0.35
Gemma-2-9B	0.003 – 0.012	0.40 – 0.70	If mag < 0.003: reduce alpha to 0.15
Qwen-2.5-7B	0.004 – 0.012	0.45 – 0.75	Usually no change needed
Phi-3-mini	0.006 – 0.020	0.55 – 0.85	Strong residuals — alpha=0.20 is safer

■ Save all `residual_profile_*.json` files permanently. They become Section 3 of the paper and inform all subsequent experiment configurations.

Pass Criterion

Profiles generated for all models. Mean `mag_mean` > 0.002 for at least 80% of layers. If any model fails this, it is not a viable NES carrier at NF4.

Experiment 3 — Clean BER Baseline (Zero-Noise Recovery)

Goal: Confirm BER = 0.000 on all target models under ideal (no-noise) conditions. This is the fundamental correctness check — if this fails, nothing else matters.

Run Procedure

```
# scripts/exp3_clean_ber.py
from src.embedding.intelligent_embedder import IntelligentEmbedder
from src.extraction.decrypt_pipeline import DecryptPipeline
from src.core.types import EmbeddingConfig

MODELS = [( 'TinyLlama/TinyLlama-1.1B-Chat-v1.0', 'llama', 32),
( 'mistralai/Mistral-7B-v0.3', 'mistral', 32),
( 'google/gemma-2-9b', 'gemma', 42),
( 'Qwen/Qwen2.5-7B', 'qwen', 28)]
MESSAGE = 'NES multi-model steganography validation.'

for model_id, family, n_layers in MODELS:
nf4, fp16, _ = load_model_pair(model_id)
residuals = extract_residuals(nf4, fp16, family)
```

```

config = EmbeddingConfig(total_payload_bits=2_000,
model_family=family, num_hidden_layers=n_layers)
embed_result = IntelligentEmbedder(config).embed(MESSAGE, residuals)
recovered, s = DecryptPipeline(key=embed_result.key).run(
embed_result.embedded_residuals, embed_result.carrier_indices)
ok = recovered == MESSAGE
print(f'{model_id} BER={0 if ok else "FAIL"} success={s["success"]}')
assert ok, f'BER > 0 on {model_id}!'

```

Expected Results

- Every model: BER = 0.000, recovered message matches exactly
- Embed time < 3 s per model on GPU (pure tensor ops, no forward pass)
- bits_embedded == total_payload_bits for all models

Common Failure Modes

- **Sign flip:** BER > 0 after embed/extract cycle:
 - Check embedded_residuals are passed to DecryptPipeline — not the originals
 - Check carrier_indices from embed_result are reused — not recalculated
- **Length header mismatch:** Message partially recovered:
 - Verify PayloadEncoder._bits_to_uint32 reads header correctly
 - Total bits embedded must == length header + payload + IV + auth-tag

Pass Criterion

BER = 0.000 on every model with 2,000-bit payload. Gate 3 of the NES benchmark passes for all families.

Experiment 4 — Capacity Curve Per Model

Goal: Determine the maximum steganographic capacity (bits) for each model at BER = 0. Larger models have more parameters per layer and thus greater capacity.

Run Procedure

```

# scripts/exp4_capacity_curve.py
PAYLOAD_SIZES = [1_000, 10_000, 50_000, 100_000, 250_000, 500_000, 1_000_000]

for model_id, family, n_layers in MODELS:
    residuals = extract_residuals(...)
    total_capacity = sum(r.numel() for r in residuals.values())
    print(f'{model_id}: capacity = {total_capacity:,} bits')

    for n_bits in PAYLOAD_SIZES:
        if n_bits > total_capacity * 0.9:
            print(f' {n_bits:>9,} bits — SKIPPED (exceeds 90% capacity)')
            continue
        config = EmbeddingConfig(total_payload_bits=n_bits, model_family=family,
num_hidden_layers=n_layers)
        result = IntelligentEmbedder(config).embed('A'*(n_bits//8), residuals)
        rec, st = DecryptPipeline(key=result.key).run(
result.embedded_residuals, result.carrier_indices)
        ber = 0.0 if (st['success'] and len(rec) > 0) else 1.0
        print(f' {n_bits:>9,} bits — BER={ber:.4f}')

```

Expected Results

Model	Total Carrier Capacity	Expected Max Payload @ BER=0
TinyLlama-1.1B (32L)	~760 M	500,000+ bits (confirmed)

Model	Total Carrier Capacity	Expected Max Payload @ BER=0
Mistral-7B (32L)	~3.8 B	> 1,000,000 bits expected
Gemma-2-9B (42L)	~4.6 B	> 1,000,000 bits expected
Qwen-2.5-7B (28L)	~3.5 B	> 1,000,000 bits expected
Phi-3-mini-4k (32L)	~2.1 B	> 500,000 bits expected

Pass Criterion

Every model achieves BER = 0 at 500,000 bits minimum. Record exact maximum per model — this becomes Table 3 in the paper.

Experiment 5 — Fidelity Validation Per Model

Goal: Measure perplexity degradation after embedding. Target: PPL degradation < 2%. This is the most important experiment for the paper's Section 5.2.

Run Procedure — Tensor Proxy (Fast, No GPU Forward Pass)

```
# scripts/exp5a_fidelity_tensor.py
from src.evaluation.fidelity_validator import FidelityValidator

for model_id, family, n_layers in MODELS:
    residuals = extract_residuals(...)
    config = EmbeddingConfig(total_payload_bits=50_000, ...)
    embed_result = IntelligentEmbedder(config).embed('A'*6000, residuals)
    validator = FidelityValidator(max_ppl_degradation=0.02)
    result = validator.validate_tensors(residuals, embed_result.embedded_residuals)
    print(result.report())
```

Run Procedure — Real PPL (Requires GPU Forward Pass)

```
# scripts/exp5b_fidelity_ppl.py
from datasets import load_dataset
dataset = load_dataset('wikitext', 'wikitext-2-raw-v1', split='validation')
texts = [t for t in dataset['text'] if len(t.strip()) > 50][:200]

for model_id, family, n_layers in MODELS:
    nf4, fp16, tok = load_model_pair(model_id)
    residuals = extract_residuals(nf4, fp16, family)
    validator = FidelityValidator(max_ppl_degradation=0.02)
    ppl_base = validator.validate_perplexity(nf4, tok, texts)
    embed_result = IntelligentEmbedder(
        EmbeddingConfig(total_payload_bits=50_000, ...)).embed('A'*6000, residuals)
    apply_residuals_to_model(nf4, fp16, embed_result.embedded_residuals, family)
    ppl_embed = validator.validate_perplexity(nf4, tok, texts)
    result = validator.compare_perplexity(ppl_base, ppl_embed)
    print(f'{model_id} {ppl_base:.3f} -> {ppl_embed:.3f} D={result.ppl_degradation*100:.3f}% {result.status}')
```

Expected Results

Model	Expected PPL Baseline	Expected After 50k Bits	Expected Delta %	Gate
TinyLlama-1.1B	12 – 14	12 – 14 (±0.05)	< 0.5%	PASS
Mistral-7B	6 – 8	6 – 8 (±0.03)	< 0.3%	PASS expected

Model	Expected PPL Baseline	Expected After 50k Bits	Expected Delta %	Gate
Gemma-2-9B	7 – 9	7 – 9 (± 0.03)	< 0.3%	PASS expected
Qwen-2.5-7B	7 – 9	7 – 9 (± 0.04)	< 0.4%	PASS expected
Phi-3-mini	5 – 7	5 – 7 (± 0.03)	< 0.3%	PASS expected

■ If PPL degradation exceeds 2% for a model: reduce alpha in AdaptiveMarginController from 0.25 to 0.15 and re-run. The margin is the primary driver of PPL degradation.

Experiment 6 — Robustness Curve Per Model

Goal: Characterize BER vs Gaussian noise sigma for each model. Determines the operational envelope and how it scales with model size.

Run Procedure

```
# scripts/exp6_robustness.py
from src.evaluation.robustness_validator import RobustnessValidator
SIGMAS = [0.0, 0.0005, 0.001, 0.002, 0.005, 0.010, 0.020]

for model_id, family, n_layers in MODELS:
    residuals = extract_residuals(...)
    config = EmbeddingConfig(total_payload_bits=10_000, ...)
    embed_result = IntelligentEmbedder(config).embed('A'*1250, residuals)
    # Get ground-truth encrypted bits for BER comparison
    msg_bits = PayloadEncoder.text_to_bits('A'*1250)
    enc_bits = AESCipher(embed_result.key).encrypt_to_bits(
        PayloadEncoder.bits_to_bytes(msg_bits))
    validator = RobustnessValidator(max_ber_at_001=0.02, max_ber_at_002=0.10, num_trials=5)
    result = validator.validate(embed_result.embedded_residuals,
        embed_result.carrier_indices, enc_bits, SIGMAS)
    print(f'{model_id}')
    print(result.report())
```

Expected Results

Model	BER @ sigma=0.001	BER @ sigma=0.002	BER @ sigma=0.005	Gate 5
TinyLlama-1.1B	0.000	0.000	0.05 – 0.15	PASS
Mistral-7B	0.000	0.000	0.02 – 0.08	PASS (larger residuals)
Gemma-2-9B	0.000	0.000	0.02 – 0.10	PASS expected
Qwen-2.5-7B	0.000	0.000	0.03 – 0.10	PASS expected
Phi-3-mini	0.000	~0.001	0.05 – 0.12	PASS expected

■ Larger models (7B+) are expected to outperform TinyLlama at noise robustness because their down_proj weights are significantly larger, producing higher-SNR residuals.

Experiment 7 — Security (Adversarial Detection) Per Model

Goal: Confirm NES embedding produces no statistically detectable signature. Target: KL divergence < 0.05 and neural detector accuracy < 55%.

Run Procedure

```
# scripts/exp7_security.py
from src.steganalysis.security_validator import SecurityValidator

for model_id, family, n_layers in MODELS:
    residuals = extract_residuals(...)
    config = EmbeddingConfig(total_payload_bits=50_000, ...)
    embed_result = IntelligentEmbedder(config).embed('A'*6000, residuals)
    validator = SecurityValidator(max_kl_divergence=0.05,
    max_detector_accuracy=0.55)
    result = validator.validate(
    original_residuals=residuals,
    embedded_residuals=embed_result.embedded_residuals,
    carrier_indices=embed_result.carrier_indices,
    )
    print(result.report())
    print(f' Gate status: {result.status}')
```

Expected Results

Model	Expected KL Divergence	Expected Detector Accuracy	Gate 6
TinyLlama-1.1B	< 0.01 (confirmed)	50.04% (confirmed)	PASS
Mistral-7B	< 0.02	49.5 – 50.5%	PASS expected
Gemma-2-9B	< 0.02	49.5 – 50.5%	PASS expected
Qwen-2.5-7B	< 0.02	49.5 – 51.0%	PASS expected
Phi-3-mini	< 0.03	50.0 – 51.5%	PASS expected

■ If KL divergence exceeds 0.05: reduce alpha (less embedding magnitude) or reduce payload density. The security/capacity tradeoff is the key tension to characterize for the paper.

Experiment 8 — Cross-Model Ablation Table

Goal: Produce a unified results table comparing all model families across all 6 quality gates. This is the primary output for Section 5 of the research paper.

```
# scripts/exp8_cross_model_table.py
from src.evaluation.nes_benchmark import NESBenchmark
import json

results = {}
for model_id, family, n_layers in MODELS:
    residuals = extract_residuals(...)
    bench = NESBenchmark(n_layers=n_layers, payload_bits=50_000,
    verbose=False, residuals=residuals)
    results[model_id] = bench.run_all()

with open('results/cross_model_table.json', 'w') as f:
    json.dump(results, f, indent=2)
```

Expected Output — Paper Table

Model	G2 QACI	G3 BER	G4 PPL Delta	G5 BER@0.001	G6 KL	G6 Det%	Result
TinyLlama-1.1B	PASS	0.000	< 0.1%	0.000	< 0.01	50.04%	PASS
Mistral-7B	PASS	0.000	< 0.1%	0.000	< 0.02	~50%	Expected PASS

Model	G2 QACI	G3 BER	G4 PPL Delta	G5 BER@0.001	G6 KL	G6 Det%	Result
Gemma-2-9B	PASS	0.000	< 0.2%	0.000	< 0.02	~50%	Expected PASS
Qwen-2.5-7B	PASS	0.000	< 0.1%	0.000	< 0.02	~50%	Expected PASS
Phi-3-mini	PASS	0.000	< 0.3%	0.000	< 0.03	~50%	Expected PASS
DeepSeek-R1	PASS	0.000	< 0.1%	0.000	< 0.02	~50%	Expected PASS

Experiment 9 — GPTQ and AWQ Quantization Variants

Goal: Test whether NES works with GPTQ and AWQ quantized models — increasingly common deployment formats. NES currently assumes NF4 via BitsAndBytes.

Key Difference

GPTQ and AWQ store weights in INT4/INT8 rather than NF4. The dequantization function differs and must be handled separately in loader.py.

```
# src/models/loader.py — ADD GPTQ support
def extract_residuals_gptq(gptq_model, fp16_model, family: str) -> dict:
    """For AutoGPTQ quantized models — qweight, scales, qzeros"""
    # GPTQ layers expose .qweight, .qzeros, .scales, .g_idx
    # Use QuantLinear.dequantize_weight() if available
    # Fallback: compare fp16 to outputs numerically
    ...

# Models to test:
# TheBloke/Mistral-7B-v0.1-GPTQ — standard GPTQ format
# TheBloke/Llama-2-7b-AWQ — AWQ format
# Qwen/Qwen2.5-7B-Instruct-AWQ — production AWQ
```

Expected Results

- GPTQ: residuals exist and are non-zero (GPTQ rounding errors comparable to NF4)
- AWQ: residuals typically smaller in magnitude — reduce alpha to 0.10–0.15
- BER = 0 achievable for both formats under clean conditions

6. Running Order and Success Gate Summary

Follow this sequence exactly. Each experiment must pass its gate before the next begins.

Step	Experiment	Script	Success Gate	Time
1	Apply code changes (Sec 4)	Manual edits	test_qaci_pipeline.py and test_phase3.py still pass	2–4 hr
2	Exp 1: Architecture Probe	exp1_probe.py	Residuals extracted, QACI runs	30 min/model
3	Exp 2: Residual Fingerprint	exp2_residual_fingerprint.py	JSON profiles saved, mag > 0.002	1 hr/model
4	Exp 3: Clean BER	exp3_clean_ber.py	BER = 0.000 all models	20 min/model
5	Exp 4: Capacity Curve	exp4_capacity_curve.py	Max capacity identified per model	2 hr/model
6	Exp 5: Fidelity	exp5a/b_fidelity.py	PPL degradation < 2%	3 hr/model GPU
7	Exp 6: Robustness	exp6_robustness.py	BER < 2% at sigma=0.001	1 hr/model
8	Exp 7: Security	exp7_security.py	KL < 0.05, detector < 55%	1 hr/model
9	Exp 8: Cross-model Table	exp8_cross_model_table.py	All models pass all gates	4 hr total
10	Exp 9: GPTQ/AWQ Variants	exp9_alternative_quant.py	BER = 0 on GPTQ and AWQ	4 hr total

7. Troubleshooting — Known Failure Modes

7.1 CUDA Out of Memory When Loading Two Models

extract_residuals() loads both NF4 (GPU) and FP16 (CPU) copies. For 7B models this requires ~8 GB GPU + ~14 GB CPU RAM. If OOM:

- Load FP16 model in bfloat16: torch_dtype=torch.bfloat16
- Extract residuals one layer at a time and delete tensors immediately
- Use offload_folder parameter in from_pretrained() for disk offloading

7.2 ModuleNotFoundError for Layer Names

If get_layer_module() raises AttributeError, the model uses a non-standard path.

```
# Debug: print all named modules to find the correct path
for name, module in model.named_modules():
    if 'mlp' in name.lower() or 'ffn' in name.lower():
        print(name, type(module).__name__)
```

Then add the correct path to MODEL_REGISTRY in src/models/registry.py.

7.3 BER > 0 After Embedding on a New Model

Most common cause: residuals have near-zero values at carrier positions, making sign-based recovery unreliable.

- Check mean mag_mean from Exp 2 — if < 0.002, increase alpha to 0.35
- Check AdaptiveMarginController.min_margin >= 1e-3 for small-residual models
- Verify embedded_residuals are written back before extraction — not the originals

7.4 KL Divergence Exceeds 0.05

- Reduce alpha (embedding magnitude): try 0.10 instead of 0.25
- Reduce payload density: embed 20% fewer bits per layer
- Increase gamma to concentrate bits in fewer, higher-quality layers

7.5 PPL Degradation > 2%

- Reduce alpha to reduce margin sizes
- Check sign_flip_rate in FidelityResult — if > 5%, QACI is selecting poor carriers
- Re-run Exp 2 and verify quality scores are meaningful (mean > 0.4) for this model

8. What Each Experiment Contributes to the Paper

Paper Section	Data Source	Claim Enabled
Abstract	Exp 3 + Exp 8	NES achieves BER=0 across N open-source LLM families
Sec 4.1 Architecture	Exp 1 + 2	NES is architecture-agnostic; requires only NF4 quantization
Table 3: Fidelity	Exp 5	PPL degradation < 0.5% across all tested model families
Table 4: Capacity	Exp 4	Per-model carrier capacity scales with parameter count
Figure 2: BER curve	Exp 6	Robustness envelope per model — operational sigma characterised
Sec 5.3: Security	Exp 7	Neural adversary at chance accuracy on all model families
Table 6: Cross-model	Exp 8	Unified 6-gate pass on 5+ model families
Appendix A: GPTQ/AWQ	Exp 9	NES generalises beyond NF4 to mainstream deployment formats

Appendix A — Models That May Not Work and Why

A.1 Full FP16 Models (Unquantized)

NES relies on quantization residuals. Without quantization there are no residuals to embed into. Full FP16 models can be used only if you apply quantization on-the-fly, but the embedded weights cannot then be saved as FP16.

A.2 GGUF Models (llama.cpp)

GGUF models run in llama.cpp, not PyTorch. Residual extraction is not straightforward — weights are not directly accessible as PyTorch tensors. NES does not currently support GGUF. Flag as future work in the paper.

A.3 Mixture-of-Experts (Mixtral, DeepSeek-MoE)

MoE models store expert weights inside each layer. The carrier extraction must target individual expert down_proj weights. The MODEL_REGISTRY entry for 'mixtral' must resolve to block_sparse_moe.experts[k].w2 for expert k. Begin with 2–3 experts only.

A.4 Models with Extremely Low Residuals

Some models (heavily regularized training) produce NF4 residuals with mean < 0.001. These require increasing the embedding margin significantly, which pushes PPL degradation beyond the 2% gate. Identify these in Experiment 2 and exclude with a note in the paper.

Appendix B — GPU and Memory Requirements

Model Size	GPU VRAM (NF4)	System RAM (FP16 CPU)	Recommended Hardware
1–2B (TinyLlama)	~2 GB	~4 GB	Any RTX 3060+ or Colab T4
7–9B (Mistral, Gemma, Qwen)	~5–6 GB	~14–18 GB	RTX 3090 / A10G / Colab A100
13B (Llama-2-13B)	~8 GB	~26 GB	RTX 4090 / A100 40GB
70B+	~35+ GB	~140 GB	A100 80GB or H100 — not for initial experiments

■ Always run on TinyLlama first after any code change to confirm the baseline still passes before touching larger models. This saves significant GPU time.